ASHISH RANJAN

Comprehensive Game Development & Technical Art Dossier

Systems Architecture · Unreal Engine 5 Engineering · 3D Asset Pipelines · Multiplayer Replication · Technical Artistry

ashish.mitblr2023@learner.manipal.edu | +91 7518105669 | Bengaluru, India | github.com/SheesshhOfficial | linkedin.com/in/sheesshh

1. Executive Profile & Technical Philosophy

Ashish Ranjan is a Computer Science & Engineering undergraduate at **Manipal Institute of Technology (MIT), Bengaluru** (Class of 2027), specializing in **Gameplay Systems Programming** and **Technical Art**. He operates at the critical intersection of low-level game engine engineering and end-to-end 3D content authoring pipelines.

Unlike developers focused solely on isolated high-level game logic or purely aesthetic asset production, Ashish bridges both disciplines:

- **Engineering Depth:** Proficient in C++ and Unreal Engine 5 (UE 5.7), specializing in client-server network replication, Server/Multicast RPCs, dynamic actor lifecycle management, Enhanced Input systems, low-level audio buffer streaming, and performance profiling.
- **Technical Art & Pipeline Rigor:** Master of Blender-to-Unreal 3D pipelines, including polygonal hard-surface modeling, game-ready topology optimization, UV unwrapping, PBR texture baking, custom skeletal armature rigging, weight painting, Inverse Kinematics (IK), and modern UE5 IK Retargeters.
- **Core CS Foundations:** Solid command of Object-Oriented Programming (certified by the *University of Pennsylvania*), Data Structures and Algorithmic Problem Solving (certified by *Meta*), Linear Algebra (quaternions, vectors, dot/cross products, matrix transforms), and Operating Systems / Memory Management.

2. Technical Competency Matrix

Domain	Core Technologies, Frameworks & Systems
Game Engines	**Unreal Engine 5 (UE5.0 – UE5.7)**, Unreal Editor Architecture, Gameplay Framework (`AGameModeBase`, `AGameStateBase`, `APlayerController`, `APawn`, `ACharacter`, `UActorComponent`).
Programming Languages	**C++ (Modern C++17/20, UE Macros, UObject System, Garbage Collection, Smart Pointers)**, Blueprints (Visual Scripting, Interfaces, Macro Libraries), Python, Java, SQL, C.
Multiplayer & Networking	**Server-Authoritative Client-Server Replication**, Server/Client/Multicast RPCs, `DOREPLIFETIME`, RepNotify (`ReplicatedUsing`), Actor Relevancy, Advanced Steam Sessions Plugin, Lobby Matchmaking.
Audio & Systems C++	**Custom C++ Blueprint Function Libraries (`UVOIPLIB`)**, `IOnlineSubsystem`, `IOnlineVoicePtr`, Raw Audio Packet Buffers, MetaSounds, 3D Spatial Audio Attenuation.
3D Digital Content Creation	**Blender (Hard-Surface Modeling, Low-Poly Retopology)**, UV Unwrapping & Packing, PBR Texture Baking (Albedo, Roughness, Metallic, Normal, AO), Draw-Call Budgeting.
Rigging & Animation	**Custom Armature Construction**, Bone Hierarchy Design, Vertex Weight Painting, 2-Bone IK Constraints, Pole Targets, UE5 Skeleton Integration, IK Rigs & IK Retargeters.
Input & Gameplay Systems	**Enhanced Input System** (`UInputMappingContext`, `UInputAction`), Runtime Context Swapping, Dynamic Collision Presets, Trace Channels, SpringArm & Camera Lag Dynamics.
Physics & UI/UX	**Physics Asset Tool (PhAT)**, Ragdoll Simulation, Line Traces / Sweeps, **UMG (Unreal Motion Graphics)**, Slate Concepts, Dynamic Reticles, Server Browsers, HUD Telemetry.
DevOps & Tooling	Git, GitHub, Perforce / VCS concepts, Visual Studio 2022, RenderDoc, Unreal Insights, Package Cooking & Shipping Builds.

3. Deep Dive: Prop Hunt Multiplayer Game System (UE 5.7)

3.1 System Overview & Asymmetric Game Architecture

The **Prop Hunt Multiplayer Game** is a complete, networked asymmetric hide-and-seek title built in **Unreal Engine 5.7**. Players are dynamically segregated into two competing teams with diametrically opposed objectives, mechanics, and input profiles:

- **Hunters:** Armed with weapons, tasked with seeking out disguised players within strict round timers. Penalized (health loss or weapon cooldowns) for firing at legitimate environmental objects rather than disguised players.
- **Props (Hiders):** Disguised as inanimate environmental props. Tasked with evading discovery until the round clock expires, navigating spatial layouts, blending naturally into room compositions, and utilizing periodic taunts.

3.2 Real-Time Mesh Morphing & Dynamic Collision Architecture

The core technical feat of the Prop player system is real-time environmental transformation:

- **Dynamic Mesh Swapping:** When a Prop player targets an environmental object and initiates a disguise action, the system performs a multi-stage line trace on custom trace channels (`ECC_PropInteractable`). Upon server validation, the character's active mesh is dynamically swapped using a `UStaticMeshComponent`.
- **Collision Recalculation & Bounds Fitting:** Inanimate objects possess vastly different dimensions than a humanoid capsule. Ashish engineered a dynamic collision resolution algorithm that dynamically queries the targeted static mesh's bounding box (`GetLocalBounds`), recalculates the `UCapsuleComponent` half-height and radius, and offsets the mesh relative to ground level to prevent the player from falling through geometry or snagging on ceilings.
- **Center-of-Mass & Physics Stabilization:** Custom physics profiles are set at runtime (`BlockAllDynamic`, `IgnorePawn` where appropriate) to prevent severe engine physics glitches or explosive angular impulses when morphing inside tight spatial crevices.
- **Camera Boom Decoupling:** When morphed into a static object, standard character rotation causes unnatural object pivoting. Ashish decoupled the camera spring arm rotation from pawn orientation, enabling 360-degree free-look reconnaissance while the physical prop mesh remains rigidly anchored to floor alignments.

3.3 Multiplayer Replication, Server Authority & Network Security

To ensure zero client-side manipulation and smooth replication across varying pings:

- **Server-Authoritative State Machine:** Transformation requests are never executed directly on the client. The client issues a validated Server RPC (`Server_RequestMorph(TargetStaticMesh)`). The server verifies player distance, validity of the targeted mesh, and team role before updating replicated actor state.
- **Replication Variables & RepNotify:** Core state variables (active mesh pointer, disguised state, health, team identifier) are marked with `UPROPERTY(ReplicatedUsing=OnRep_MeshChanged)`. When the server mutates the state, `OnRep` handlers execute seamlessly on all autonomous and simulated proxies, updating local visuals without redundant RPC spam.
- **Multicast Event Execution:** Audio-visual effects (poof particle emitter, disguise sound attenuation) are fired via `NetMulticast` unreliable RPCs to conserve network bandwidth while ensuring all nearby players perceive the transformation in real time.
- **Server-Side Hit Detection:** To prevent client-side aim exploitation, Hunter weapon fire is validated on the server via line traces, cross-referencing server-side actor locations and lag compensation timestamps.

3.4 Steam Online Subsystems & Custom C++ Voice Pipeline (`UVOIPLIB`)

- **Advanced Steam Sessions Integration:** Configured the Steamworks SDK and Unreal Engine's Online Subsystem Steam (`OSS_Steam`). Implemented full lobby matchmaking, server discovery with custom session settings (Map Name, Active Players, Max Players, Game State), and direct Steam friend invites.
- **Custom C++ Voice Library (`UVOIPLIB`):** Ashish authored a custom C++ Blueprint Function Library interfacing directly with `IOnlineSubsystem::Get()->GetVoiceInterface()`. This custom library exposed low-level audio packet buffer monitoring, voice capture activity states, and automated mute/unmute routing based on player life state (e.g., dead players routed to a separate spectating voice channel to prevent ghost-calling).

3.5 Enhanced Input System Integration

Rather than relying on legacy input mappings, the project leverages Unreal Engine's **Enhanced Input System**:

- **Dynamic Context Switching:** Separate `UInputMappingContext` (IMC) data assets were established for `IMC_Hunter` and `IMC_Prop`. When a player spawns or switches roles, the local `UEnhancedInputLocalPlayerSubsystem` flushes prior contexts and applies the targeted mapping dynamically.
- **Specialized Prop Input Actions:** Props feature unique Input Actions including:
  - *IA_RotationLock:* Toggles pawn rotation synchronization with camera yaw, allowing players to angle themselves naturally against walls and freeze.
  - *IA_Taunt:* Triggers a localized spatial audio whistle or joke to grant bonus score while revealing approximate proximity to hunters.
  - *IA_Hop:* Applies a normalized upward impulse for agile environmental parkour.

3.6 Debugging, Memory Optimization & Blueprint Performance Traps

During development, Ashish diagnosed and resolved critical engine bottlenecks:

- **Dynamic Actor Spawning Garbage Collection Traps:** Early iterations dynamically spawned new actor instances upon disguise, resulting in severe garbage collection hitching and dangling reference crashes. Ashish refactored the system to mutate component state on the existing pawn rather than destroying/re-spawning actors.
- **Tick Function Elimination:** Replaced expensive per-frame `Tick` calculations in Blueprints with asynchronous event timers, property delegates, and Blueprint Interfaces (`BPI`), locking solid 120 FPS performance in crowded multiplayer environments.

4. Deep Dive: End-to-End 3D Asset Pipeline (Blender to UE5)

4.1 Pipeline Architecture & Philosophy

A high-performance real-time game relies fundamentally on clean asset hygiene. Ashish designed and standardized an end-to-end 3D content creation pipeline that guarantees visual fidelity while adhering strictly to rendering draw-call and poly-count budgets.

4.2 3D Modeling, Topology & Polygon Budgeting

- **Hard-Surface Modeling in Blender:** Authoring clean geometric assets with intentional edge loops, non-manifold geometry elimination, and planar face preservation.
- **Bevel Modifiers & Weighted Normals:** Applying non-destructive bevel modifiers with weighted normal calculations (`Set Split Normals`), achieving high-poly baked aesthetic appearances on lightweight low-poly geometry without increasing texture footprint.
- **Silhouette & LOD Planning:** Ensuring distinct silhouette readability at long distances while budgeting polygon densities to under target triangle limits per asset class.

4.3 UV Unwrapping, Packing Efficiency & Texel Density

- **Seam Placement Strategy:** Placing UV seams strategically along natural geometric crevices, sharp angles, and undersides to obscure texture seams from the player's primary line of sight.
- **Texel Density Standardization:** Enforcing uniform texel density across asset libraries (e.g., 512px/meter for background props, 1024px/meter for interactive objects) to prevent visual dissonance between adjacent meshes.
- **UV Packing Optimization:** Maximizing UV island packing efficiency ($> 82\%$ coverage), minimizing wasted texture padding while maintaining sufficient margin to prevent mipmap bleeding artifacts.

4.4 Diagnosing & Eliminating Classic Industry Pipeline Failure Modes

Through hands-on iteration, Ashish cataloged and engineered definitive solutions for common 3D pipeline pitfalls:

Pipeline Defect	Root Cause Breakdown	Implemented Technical Solution
Inverted / Warped Normal Maps	Discrepancy between OpenGL (Blender's default $+Y$ green channel) and DirectX (Unreal Engine's default $-Y$ green channel).	Configured normal texture bake scripts with inverted green channel or checked `Flip Green Channel` in UE5 texture settings.
"White Material" / Missing Shaders	FBX exporter failing to embed absolute texture paths or UE5 material auto-generation misassigning PBR slots.	Established standard Master Materials in UE5; authored streamlined naming conventions (`M_Master`, `MI_Asset_01`) with automatic parameter binding.
Scale & Orientation Disorientation	Blender's native coordinate frame (Z-Up, -Y Forward) exported without unit normalization to UE5's centimeter scale.	Standardized Blender scene units to Centimeters (0.01 scale); enforced `Apply All Transforms` and standardized FBX export presets (`!FBX -Z Forward / Y Up`).
High Draw Call Overhead	Multiple material slots assigned to single static meshes causing separate draw call dispatches per frame.	Consolidated multi-material meshes into unified texture atlases and single material slot assignments prior to in-engine export.

5. Deep Dive: Character Rigging, Inverse Kinematics & Animation Setup

5.1 Custom Armature Construction in Blender

- **Hierarchical Bone Architecture:** Built custom anatomical and stylized skeletal armatures adhering to standard naming conventions (`Root`, `Pelvis`, `Spine_01..03`, `Clavicle`, `UpperArm`, `LowerArm`, `Hand`, `Thigh`, `Calf`, `Foot`).
- **Bone Roll Alignment:** Strictly calibrated bone roll axes across symmetrical limbs to ensure rotation Euler angles operate reliably during complex pose calculations.

5.2 Weight Painting & Mesh Deformation Preservation

- **Joint Volume Preservation:** Manually painted vertex weights across shoulder, elbow, hip, and knee articulations, counteracting standard linear blend skinning volume loss ("candy-wrapper" mesh pinching).
- **Deformation vs. Control Bone Segregation:** Explicitly separated deformation bones (exported to UE5) from control helper bones and mechanism bones, keeping in-engine skeletal payloads compact.

5.3 Inverse Kinematics (IK) & Modern UE5 Retargeting Pipeline

- **2-Bone IK Solvers:** Configured procedural IK constraints in Blender with pole targets to control knee and elbow bending planes accurately.

- **Unreal Engine 5 IK Rig & IK Retargeter Setup:** Rather than relying on deprecated legacy retargeting systems, Ashish configured UE5's modern **IK Rig** system:
  - Authored custom `IK_Source` and `IK_Target` asset rigs, defining clean bone chains for Spine, Limbs, and Head.
  - Configured an **IK Retargeter** asset, aligning base poses, adjusting translation retargeting to `Skeleton` or `Animation Scaled`, and retargeting complete animation libraries (runs, jumps, crouches, idle blends) with zero joint distortion.

5.4 Physics Asset Tool (PhAT), Sockets & Ragdoll Engineering

- **PhAT Ragdoll Setup:** Constructed physical bodies using spheryl, capsule, and box shapes tailored to character anatomy. Configured angular joint constraints (Twist, Swing1, Swing2) with strict physical limits to prevent unnatural limb hyperextension during ragdoll states.
- **Kinematic-to-Simulated Blending:** Engineered dynamic death triggers that smoothly interpolate character bones from keyframed animation states into full physical ragdoll simulations (`SetAllBodiesBelowSimulatePhysics`).
- **Socket Attachments:** Defined and calibrated skeletal sockets (`Hand_R_Socket`, `Spine_Back_Socket`) with custom rotation offsets for seamless runtime weapon and prop snapping.

6. Deep Dive: Advanced Gameplay Systems & Audio Exploration

6.1 Proximity Audio, Spatialization & MetaSounds

Audio is 50% of player immersion and a critical mechanical pillar in hide-and-seek games.

- **3D Audio Spatialization Curves:** Configured custom attenuation settings utilizing natural logarithmic falloff curves, ensuring distant sounds decay smoothly while close-range taunts provide distinct directional stereo/binaural imaging.
- **Occlusion Filtering:** Implemented real-time line trace occlusion checks between the listener and sound origin, applying low-pass frequency filtering (muffling) when physical walls or obstacles obstruct direct line of sight.
- **Taunt Mechanic System:** Created dynamic audio dispatchers that cycle through randomized audio soundcues with cooldown timers, broadcasting positional sound cues across the network to provide hunters auditory clues.

6.2 Event-Driven Frameworks vs. Frame-Rate Optimization

Ashish adheres to strict production optimization guidelines:

- **Minimizing Blueprint `Tick` Execution:** `Tick` executions are strictly disabled on static or state-driven actors. Instead, game loops leverage dynamic timers (`SetTimerByEvent`), event dispatchers, and delegate callbacks.
- **Decoupled Blueprint Interfaces (`BPI`):** Systems communicate via interfaces (`BPI_Interactable`, `BPI_Damageable`) rather than direct hard object casting. This prevents cascading asset loading into memory and guarantees modularity.

7. Computer Science & Applied Mathematics in Engine Engineering

Ashish grounds his gameplay development in rigorous computer science and applied mathematics:

7.1 3D Vector Calculus & Trigonometry

- **Dot Product ($\mathbf{A} \cdot \mathbf{B}$):** Utilized for directional field-of-view (FOV) checks, determining whether a Hunter is actively facing a disguise prop ($\cos\theta >$ threshold), and calculating lighting/shading surface angles.
- **Cross Product ($\mathbf{A} \times \mathbf{B}$):** Utilized for calculating surface normal orthogonal vectors, determining climbable surface normals, and orienting props dynamically to sloping terrain.
- **Quaternions & Rotators:** Employed to eliminate Gimbal Lock during complex 3D character rotations, interpolating rotations smoothly using Spherical Linear Interpolation (`SLerp` / `RInterpTo`).

7.2 Spatial Tracing & Physics Raycasting

- **Single & Multi-Line Traces:** Executing high-efficiency line traces using custom object response channels (`ECC_GameTraceChannel`) to perform hitscan weapon ballistics and prop surface detection without testing against unnecessary scene colliders.
- **Sweep Traces:** Utilizing sphere and capsule sweeps to validate that a proposed prop disguise mesh will not intersect with existing static geometry before granting transformation approval.

7.3 Physics Bounding Box & Center-of-Mass Mathematics

To guarantee that transformed pawns behave physically identically to real environmental meshes without clipping through floors:

- Given an extracted static mesh bounding box with extents (E_x, E_y, E_z), the dynamic capsule dimensions are computed as:

$$\text{CapsuleRadius} = \frac{1}{2} \cdot \max(E_x, E_y), \qquad \text{CapsuleHalfHeight} = \max(E_z, \text{CapsuleRadius})$$

- A vertical pawn origin translation vector $\mathbf{v}_{\text{offset}} = (0, 0, -\text{CapsuleHalfHeight})$ is applied dynamically to keep the collision body anchored perfectly flush against terrain surfaces.

7.4 Multiplayer Hit Validation & Latency Rollback Algorithms

- **Client Timestamp Historical Buffers:** The dedicated server maintains a rolling circular buffer of player collision capsule transforms indexed by engine tick timestamps.
- **Lag Rollback Validation:** When a Hunter fires a shot at time t_{client}, the server calculates the one-way trip latency $\tau = \text{RTT}/2$, rewinds target bounding boxes to $t_{\text{rewind}} = t_{\text{server}} - \tau$, executes the raycast intersection check, and restores the current world state, guaranteeing responsive hit registration under varying network conditions.

8. Shaders, Production Workflows & Studio Readiness

8.1 Unreal Engine Master Material & Shader Graph Architecture

To achieve high-end visual fidelity with minimum draw-call overhead, Ashish authors centralized Master Materials:

- **Packed ORM Texture Streaming:** Consolidates Ambient Occlusion (Red), Roughness (Green), and Metallic (Blue) channels into a single texture asset, cutting texture sampler counts by 66%.
- **Dynamic Normal Intensity:** Incorporates `FlattenNormal` nodes with runtime scalar parameters to modulate surface bumps without re-baking textures.
- **Custom Stencil Outline Post-Processing:** Utilizes the Custom Depth / Custom Stencil pass in UE5 post-process materials to render crisp, glowing outlines on targeted interactive props.

8.2 Production Version Control & Binary Asset Hygiene

- **Git LFS (Large File Storage):** Configured custom `.gitattributes` filters to track binary Unreal Engine assets (`*.uasset`, `*.umap`) and 3D modeling files (`*.blend`, `*.fbx`, `*.psd`), preventing repository bloat.
- **File Locking Standards:** Enforces exclusive file locking protocols on maps and Blueprints during collaborative game jams, eliminating unmergeable binary asset conflicts.

8.3 Future Upgrades & Portfolio Roadmap

As documented in the master engineering backlog (`future_upgrades/roadmap.md`):

- **itch.io Playable Builds:** Compiling shipping builds and packaging WebGL / standalone executables for direct public gameplay evaluation.
- **ArtStation & Sketchfab 3D Turnarounds:** Integrating interactive WebGL 3D model turnaround showcases highlighting topology, wireframes, and UV layouts.
- **Dedicated Linux Server Deployment:** Packaging headless Unreal Engine dedicated server builds for cloud containerization (Docker) and AWS GameLift / PlayFab multiplayer deployment.

8.4 Target Industry Roles

Ashish's dual skill set positions him immediately for impactful contribution across:

1. **Gameplay Programmer (Unreal Engine / C++):** Crafting robust multiplayer mechanics, weapon systems, player movement, network replication, and input architecture.
2. **Technical Artist (Rigging & Pipelines):** Bridging art and engineering, standardizing Blender-to-Unreal export pipelines, building armatures, weight painting, and configuring IK retargeters.
3. **Generalist Game Developer:** Delivering end-to-end prototypes from conceptual greyboxing to shipped, polished playable products.

Document compiled from verified engineering repositories and project artifacts. For source code access, playable demonstrations, or technical walk-throughs, contact: ashish.mitblr2023@learner.manipal.edu